Labo 12 — Réponses commentées

Chaque réponse suit le même schéma : la réponse, le mécanisme, la nuance ou le piège, un exemple vérifiable au terminal.

Question 1 — Trois ReplicaSets, trois Pods

Réponse. Trois ReplicaSets (un par version du template) et **trois** Pods, tous dans le ReplicaSet le plus récent. `kubectl get rs` montre le dernier à `3 3 3` et les deux autres à `0 0 0`. Les anciens sont conservés parce qu'ils **sont** l'historique : `rollout undo` remet simplement les réplicas d'un ancien ReplicaSet à 3 et ceux du courant à 0. `revisionHistoryLimit` (10 par défaut) dans la spec du Deployment borne leur nombre.

Pourquoi. Un ReplicaSet à zéro ne coûte qu'un objet dans etcd ; le garder rend le retour arrière instantané, sans reconstruire de template.

Nuance. Deux mises à jour qui aboutissent au **même** template (1.0 → 1.1 → 1.0) réutilisent le ReplicaSet existant : le hash du template est identique.

Exemple.

```
kubectl get rs -l app=api   # api-6c7bb54fc5 0 0 0 / api-5c8d86977d 0 0 0 / api-… 3 3 3
```

Question 2 — Selector et labels incohérents

Réponse. `The Deployment "api" is invalid: spec.template.metadata.labels: Invalid value: … selector does not match template labels`. La règle : le `selector` du Deployment **doit** sélectionner les Pods que son template produit, sinon il créerait des Pods qu'il ne reconnaît pas comme les siens. Si le cluster acceptait, le ReplicaSet verrait toujours `0` Pod `app: api`, en créerait sans fin des `app: backend` qu'il ne compte pas — une fuite de Pods.

Pourquoi. La relation Deployment → Pods n'est pas une possession mais une **sélection** ; elle doit être cohérente dès la création.

Nuance. À l'inverse, un Pod créé à la main avec `app: api` serait **adopté** par le ReplicaSet (et un Pod en trop serait supprimé) : les labels sont un contrat, pas une décoration.

Exemple.

```
kubectl apply -f deployment-mauvais.yaml 2>&1 | grep -o 'selector does not match template labels'
```

Question 3 — La séquence d'un rolling update

Réponse. Avec `maxSurge: 25 %` et `maxUnavailable: 25 %` sur trois réplicas (arrondis : 1 en plus, 0 en moins), les Events sont : nouveau RS `0 → 1` ; dès que ce Pod est prêt, nouveau `1 → 2` et ancien `3 → 2` ; puis ancien `2 → 1`, nouveau `2 → 3` ; enfin ancien `1 → 0`. À aucun moment moins de trois Pods prêts. Créer trois nouveaux Pods d'un coup demanderait six Pods (dépasse `maxSurge`) ou supprimerait trois anciens avant que les nouveaux soient prêts (dépasse `maxUnavailable`).

Pourquoi. Le contrôleur de Deployment ne fait qu'ajuster les réplicas de deux ReplicaSets, pas à pas, en respectant les deux bornes et l'état `Ready` des Pods.

Nuance. La readiness probe est ce qui rend « prêt » significatif ; sans elle, le rollout va plus vite et peut couper le service pendant le démarrage des JVM.

Exemple.

```
kubectl get events --sort-by=.lastTimestamp | grep ScalingReplicaSet | tail -6
```

Question 4 — Un seul Pod en `Error`

Réponse. Le Deployment a créé **un** Pod de la nouvelle version (`maxSurge` = 1) et attend qu'il soit prêt avant de retirer un ancien ; comme il sort en erreur en boucle, le rollout **s'arrête là**, et les trois anciens continuent de servir — c'est le filet de sécurité. `kubectl rollout status` attend puis expire : `error: timed out waiting for the condition`. Cause : `kubectl logs rs/api-6c87f96dd5 --tail=1` (ou `kubectl logs api-6c87f96dd5-4v6s6 --previous`) → `demarrage impossible : configuration manquante`. Réparation : `kubectl rollout undo deployment/api`.

Pourquoi. `maxUnavailable: 25 %` de 3 = 0 : aucun ancien Pod ne peut être retiré tant qu'un nouveau n'est pas prêt.

Nuance. Sans readiness probe, un conteneur qui **démarre** puis plante quelques secondes plus tard aurait pu passer « prêt » le temps de retirer un ancien ; la sonde ferme cette fenêtre.

Exemple.

```
kubectl get deployment api      # READY 3/3   UP-TO-DATE 1   AVAILABLE 3
kubectl rollout undo deployment/api && kubectl rollout status deployment/api
```

Question 5 — Liveness sur une dépendance

Réponse. Quand la base tombe, `/actuator/health` passe `DOWN`, la liveness échoue trois fois, le kubelet **tue et redémarre** l'API ; l'API redémarre, ne trouve toujours pas la base, est retuée : `RESTARTS` grimpe, puis `CrashLoopBackOff`. Pendant cinq minutes, l'API, qui aurait pu servir les requêtes sans base (pages statiques, cache, erreurs propres), est injoignable, et au retour de la base elle met plusieurs minutes à sortir du backoff. Il fallait une liveness sur `/actuator/health/liveness` (état interne du processus, indépendant des dépendances) et, au besoin, une **readiness** sur la base : le Pod sort du trafic sans être tué.

Pourquoi. Redémarrer ne répare que les pannes **internes** au processus ; une dépendance absente ne se répare pas par un redémarrage.

Nuance. Spring Boot sépare les deux justement pour cela : `LivenessState` ne dépend d'aucun `HealthIndicator` externe.

Exemple.

```
kubectl describe pods -l app=api | grep -E 'Liveness probe failed|Killing' | tail -2
```

Question 6 — Readiness cassée, liveness cassée

Réponse. Readiness sur `8081` : un nouveau Pod `0/1 Running` (jamais prêt), les trois anciens `1/1`, `kubectl get deployment` montre `READY 3/3 UP-TO-DATE 1 AVAILABLE 3` ; le rollout est bloqué, le service **répond** par les anciens Pods. Liveness sur `8081` : les nouveaux Pods passent `Ready` (la

readiness, elle, réussit), le rollout **avance** et remplace les trois anciens, puis la liveness tue chaque nouveau Pod : `0/1 CrashLoopBackOff` × 3, service **mort**.

Pourquoi. Le rollout n'attend que la readiness ; la liveness agit après coup, sur des Pods déjà comptés comme bons.

Nuance. C'est l'argument pour tester un changement de sonde en recette, et pour donner à la liveness un `initialDelaySeconds` et un `failureThreshold` généreux.

Exemple.

```
kubectl get pods            # readiness : api-xxx 0/1 Running + 3 anciens 1/1 ; liveness : 3 × CrashLoopBackOff
kubectl rollout undo deployment/api
```

Question 7 — `CrashLoopBackOff`

Réponse. Un état du **kubelet** : le conteneur sort en erreur, le kubelet le relance, il ressort, et le kubelet **double le délai** avant chaque relance (10 s, 20 s, 40 s… plafonné à cinq minutes) pour ne pas saturer le nœud. L'application, elle, n'a qu'un état : elle plante. La cause est dans `kubectl logs <pod> --previous`, la sortie de la tentative précédente ; `kubectl logs` sans option montre le conteneur **courant**, souvent vide s'il vient d'être relancé ou s'il n'a pas encore écrit.

Pourquoi. Le backoff est une protection du nœud, pas un diagnostic : rien dans le délai ne dit pourquoi le processus sort.

Nuance. Le compteur `RESTARTS` est remis à zéro quand le conteneur tourne dix minutes sans sortir : un redémarrage ancien peut donc ne plus se voir.

Exemple.

```
kubectl get pod api-xxx -o
jsonpath='{.status.containerStatuses[0].lastState.terminated.exitCode}'
kubectl logs api-xxx --previous | tail -3
```

Question 8 — `HEALTHCHECK` contre readiness

Réponse. Le `HEALTHCHECK` est exécuté **dans** le conteneur par le moteur (Podman via un timer, Docker via le daemon) : l'outil (`wget`, `curl`) doit être dans l'image ; un échec change l'état `(unhealthy)` et rien d'autre, sauf Compose `depends_on`. La `readinessProbe httpGet` est exécutée par le **kubelet**, depuis le nœud, par une requête HTTP vers l'IP du Pod : aucun outil dans l'image ; un échec retire le Pod du trafic. Une image distroless (sans shell ni `wget`) ne peut pas porter de `HEALTHCHECK` fonctionnel ; elle fonctionne parfaitement avec une sonde `httpGet` ou `tcpSocket`.

Pourquoi. Kubernetes a déplacé la vérification du conteneur vers la plateforme ; c'est pourquoi il ignore le `HEALTHCHECK` des images.

Nuance. La sonde `exec` reprend le modèle Docker (commande dans le conteneur) pour les cas sans port HTTP.

Exemple.

```
kubectl explain pod.spec.containers.readinessProbe.httpGet | head -8
```

Question 9 — Révisions avec des trous

Réponse. Chaque modification du template crée une révision, et chaque `rollout undo` aussi : revenir à la révision 1 **crée** une révision 3 identique à la 1 (le ReplicaSet de la 1 est réutilisé et renuméroté). Les numéros absents sont ceux que les retours arrière ont « consommés » : `1 → 3`, `2 → 4`, etc. Ce n'est pas un problème : l'historique reste cohérent, et `rollout undo --to-revision=N` fonctionne avec les numéros affichés.

Pourquoi. Une révision est une annotation `deployment.kubernetes.io/revision` sur un ReplicaSet ; un ReplicaSet réactivé reçoit le numéro suivant.

Nuance. La colonne `CHANGE-CAUSE` vide se remplit avec `kubectl annotate deployment api kubernetes.io/change-cause="passage en 1.1"`, précieux en revue d'incident.

Exemple.

```
kubectl get rs -o custom-
columns=NAME:.metadata.name,REV:.metadata.annotations.deployment\.kubernetes\.io/revision,REPLI
CAS:.spec.replicas
```

Question 10 — `maxSurge` et `maxUnavailable`

Réponse. `maxSurge: 0, maxUnavailable: 1` : le cluster retire d'abord un ancien Pod, puis crée un nouveau ; pic 3, creux **2** Pods — jamais plus de trois, donc adapté à des nœuds pleins, au prix d'une capacité réduite pendant la mise à jour. `maxSurge: 1, maxUnavailable: 0` : il crée un nouveau Pod, attend qu'il soit prêt, puis retire un ancien ; pic **4**, creux 3 — jamais moins de trois prêts, donc adapté à un service qui ne doit pas perdre de capacité, au prix d'un Pod de plus à loger.

Pourquoi. Les deux bornes se lisent comme un budget : combien je peux dépasser, combien je peux manquer.

Nuance. `maxSurge: 0, maxUnavailable: 0` est refusé : le rollout ne pourrait jamais commencer.

Exemple.

```
kubectl patch deployment api -p '{"spec":{"strategy":{"rollingUpdate":
{"maxSurge":1,"maxUnavailable":0}}}}'
```

Question 11 — Deux Pods `Pending`

Réponse. Le nœud n'a plus de **ressources** pour eux (mémoire ou CPU, si le template a des `requests`, labo 16) ou atteint sa limite de Pods ; `kubectl describe pod` montre `FailedScheduling … 0/1 nodes are available: 1 Insufficient memory`. Un rollout dans cet état avance **partiellement** : les Pods qui peuvent être planifiés sont remplacés, les `Pending` le restent (pour la nouvelle version), et `rollout status` n'aboutit jamais.

Pourquoi. Le scheduler ne place un Pod que si un nœud peut satisfaire ses demandes ; `scale` ne crée pas de capacité.

Nuance. Sans `requests`, un nœud accepte des Pods jusqu'à sa limite de nombre (110 par défaut) : les `Pending` viennent alors d'autre chose (volume, affinité, taint).

Exemple.

```
kubectl describe node | grep -A6 'Allocated resources'
```

Question 12 — Modifier un Pod à la main

Réponse. Au prochain **rollout**, le Pod est remplacé par un Pod neuf issu du template : la modification disparaît. Au prochain **redémarrage** du conteneur (liveness, crash), le système de fichiers de la couche writable est recréé depuis l'image : disparue aussi. Les **deux autres réplicas** n'ont jamais vu la modification : le service se comporte différemment selon le Pod qui répond. Il fallait mettre la configuration dans une **ConfigMap** ou un **Secret** (labo 14) référencés par le template, et laisser le rollout propager le changement à tous les réplicas.

Pourquoi. Un Pod est jetable (labo 06) ; seule la spec est durable.

Nuance. `kubectl exec` reste légitime pour **observer** et pour un diagnostic temporaire, jamais pour configurer.

Exemple.

```
kubectl rollout restart deployment/api    # recrée tous les Pods depuis le template : les bricolages disparaissent
```

Question 13 — Un Pod nu en production

Réponse. (1) **Pas d'auto-réparation** : le Pod supprimé ou son nœud perdu, il ne revient pas. (2) **Pas de mise à jour sans coupure** : changer l'image impose `delete` puis `apply`. (3) **Pas de retour arrière** : aucun historique. (4) **Pas de montée en charge** : passer à deux instances exige un second manifeste. Le seul cas légitime : un **traitement ponctuel** ou un test, qu'un `Job` fait d'ailleurs mieux ; pour un service, même à une instance, le Deployment ne coûte que cinq lignes de plus.

Pourquoi. Le Pod est une unité d'exécution, pas un objet de gestion ; les contrôleurs ajoutent la gestion.

Nuance. Un Pod nu **est** recréé par le kubelet s'il est un *static Pod* (fichier sur le nœud) — c'est ainsi que tournent les composants du control plane dans certains clusters, pas vos applications.

Exemple.

```
kubectl create deployment api --image=localhost/api:1.0 --replicas=1 --dry-run=client -o yaml | head -20
```